EcoVerse

Gamified Sustainability and Verified Environmental Action

1. Abstract and introduction

EcoVerse is a full-stack web application that links everyday environmental actions with visible personal progress. A user discovers a practical challenge, completes it outside the application and submits supporting evidence. A reviewer then approves or rejects the submission. Approved actions contribute to Eco Points, badges, a leaderboard and a digital forest. The system also provides community participation and a directory of environmental organizations, allowing users to move from individual activity toward shared participation.

The project demonstrates browser-based interface design, asynchronous API communication, relational database storage, session authentication, role-based authorization, file handling and interactive graphics. Its frontend uses HTML, CSS and JavaScript with Vite, while the backend uses Node.js, Express and MySQL. Three.js supports the forest visualization. The purpose of the project is to encourage repeatable participation and document a traceable activity history; it does not claim to independently measure or certify environmental outcomes.

1.1 Problem statement

Environmental information is easy to find, but a person may still be unsure what to do next or how to maintain a routine. A simple checklist can record a click without establishing that an action was performed. Separate resources for volunteering, personal tracking and learning can also make the experience fragmented. EcoVerse addresses this gap by bringing action discovery, evidence review, progress feedback and community discovery into one interface.

1.2 Objectives

Objective	Implementation approach
Make actions understandable	Provide categories, instructions, difficulty, estimated time and evidence requirements.
Keep rewards traceable	Award verified-action points through approved submissions and a transaction ledger.
Maintain engagement	Show forest progression, milestones, badges and category-based recommendations.
Support participation	Provide teams, campaigns, optional shared field notes and organization links.
Demonstrate full-stack skills	Connect a responsive browser interface to authenticated APIs and MySQL.

1.3 Scope and report context

The report describes the current local implementation and its frontend deployment. Screenshots were captured from the actual local application using an isolated demonstration account, not from design mockups. The college cover and student details are intentionally omitted for separate preparation. Proposed enhancements are distinguished from implemented modules.

2. Frontend: home and personal dashboard

The home page communicates the project through a direct action prompt and links to the journey. The shared navigation exposes My World, Discover, Challenges, Community and Connect. This makes the primary tasks visible while profile and supporting views remain reachable within the application.

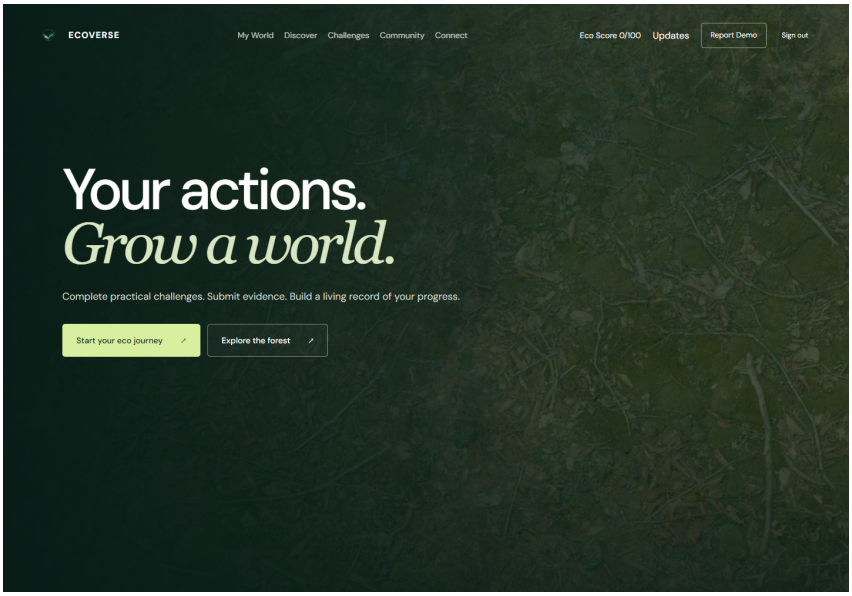


Figure 1. Actual home page captured locally with the report demonstration account.

My World summarizes the signed-in user's verified points, Eco Score, actions, rank and forest stage. Below the main summary, the interface presents activity status, milestones and recommendations. A new account starts at zero; pending evidence does not immediately become verified progress.

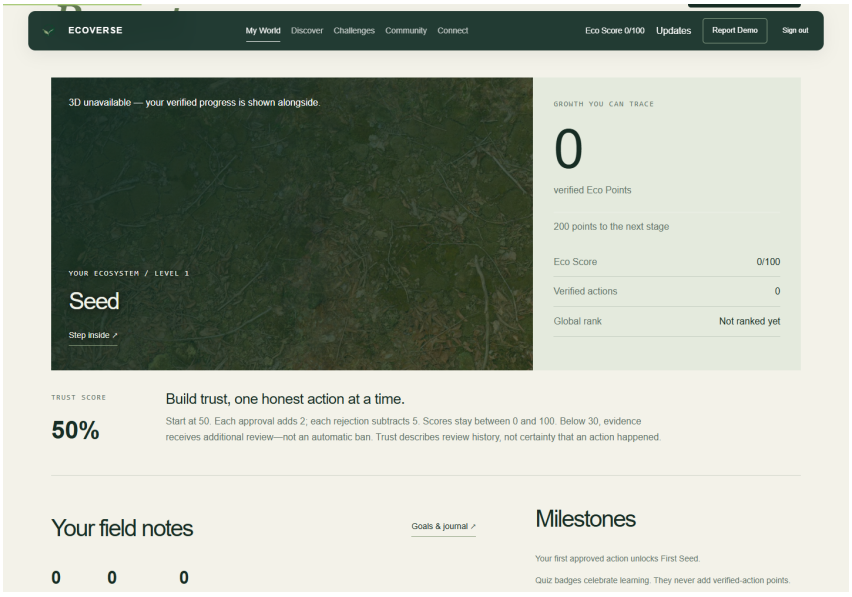


Figure 2. New-account dashboard. Any displayed graphics fallback is the actual capture environment state, not a substituted forest render.

2.1 Feedback and accessibility

Loading messages, empty states and readable action labels guide a first-time user. Status distinctions help explain why a submitted action has not yet earned points. Reduced-motion behavior is supported in parts of the interface, and a fallback keeps progress information available when WebGL cannot initialize.

2.2 Frontend: challenges and progression

The Challenges page presents available expeditions in a searchable catalog. Category filters cover energy, water, waste, transport, lifestyle and nature. Cards summarize the activity, difficulty, estimated time, proof requirement and points available after approval. Search and filtering reduce the effort of finding an action appropriate to the user's interests.

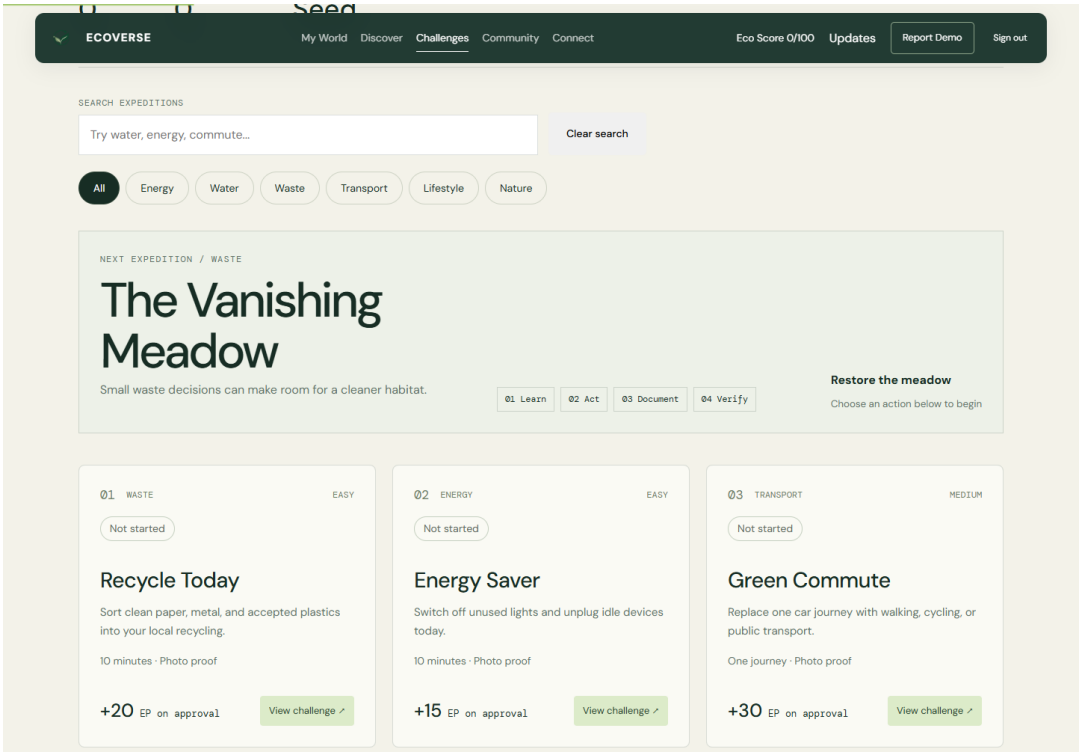


Figure 3. Challenge catalog from the running local backend. Points shown on cards are conditional on approval.

Challenge interaction and status feedback

The member starts an action, completes the task in real life and submits evidence with a description. The application distinguishes IN_PROGRESS, PENDING, APPROVED and REJECTED states. A rejected submission includes feedback for correction. Repeat eligibility and active-attempt checks are handled on the server; reward decisions are not controlled by a client-side counter.

Progression displayed to the user

Mechanic	Configured behavior
Forest stages	Five default thresholds: 0, 200, 500, 1,000 and 2,000 verified points.
Tree and wildlife unlocks	Tree count grows with points, capped at 80; wildlife unlocks in 200-point steps, capped at 12.
Milestones	First Seed rewards the first approval; Growing Strong and Forest Keeper use point thresholds.
Trust	Review history influences a trust score and review handling; it is not a guarantee of honesty.

These values are game-design rules. A digital tree does not automatically represent a real tree planted, and a forest level is not an ecological measurement. Keeping this distinction explicit avoids overstating what the application has verified.

2.3 Frontend: digital forest and habitat

The digital world provides a visual record of progress. The project includes terrain, trees, birds, butterflies and supporting environment assets. Three.js renders the scene, while progression logic controls unlock counts and growth. Time-of-day and related presentation controls personalize the view without changing verified-action history.

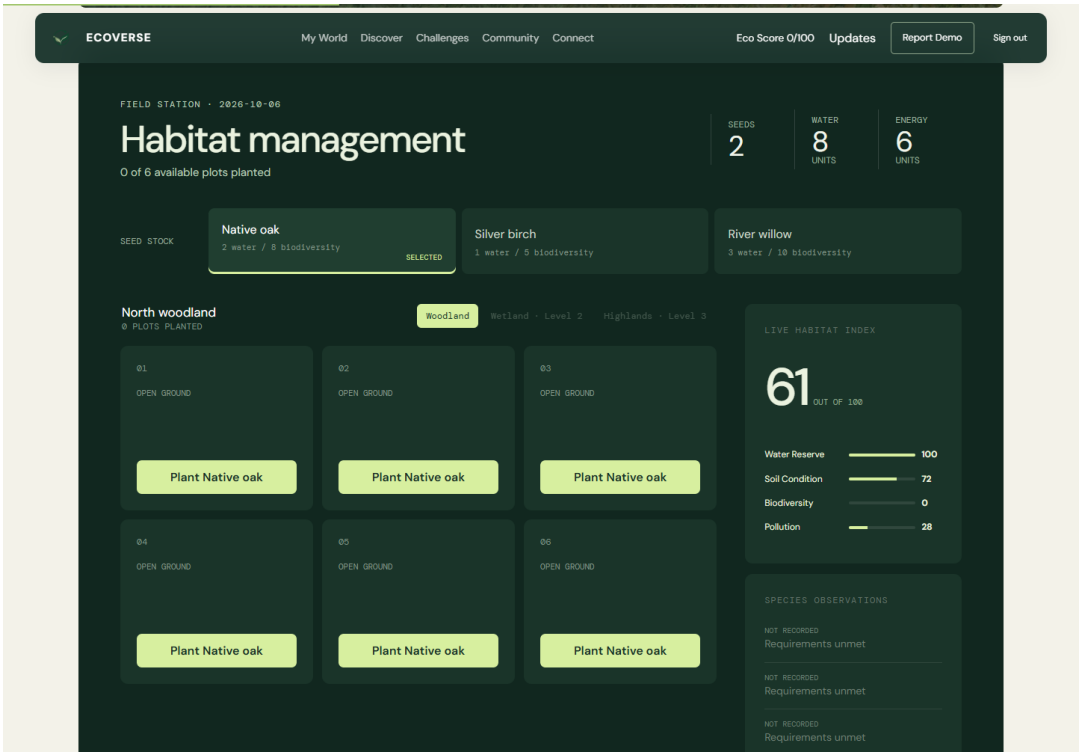


Figure 4. Habitat management on the forest page. This new demonstration account begins with open plots and limited resources.

Habitat interaction

Habitat management adds an interactive activity beyond scrolling. Players choose a species and use available resources to plant, water and clear weeds. Woodland, wetland and highland areas provide different zones, with access linked to progression. The interface presents resource quantities, plot states, environmental events and a habitat index so actions have visible consequences.

Game state and verified action

The habitat is a simulation for engagement. Its resources and condition values are distinct from evidence-backed challenge records. This allows a user to interact with the world without implying that every in-game click completes an external environmental task. Verified progress remains tied to approved submissions and the point ledger.

Performance and fallback behavior

A three-dimensional scene costs more processing time than ordinary page content. The application therefore includes graphics controls and fallback presentation, while asset tests check important geometry, animation and size characteristics. The screenshot records the state available during capture. Full visual quality and frame rate still depend on the browser and graphics hardware.

2.4 Frontend: Community and Connect

Community combines campaigns, teams and voluntarily published field notes. Members can create or join a team, react to a shared action and add comments. Publication is limited to approved actions, and sharing the evidence photograph is an explicit choice. Private submissions and public community content therefore serve different purposes.

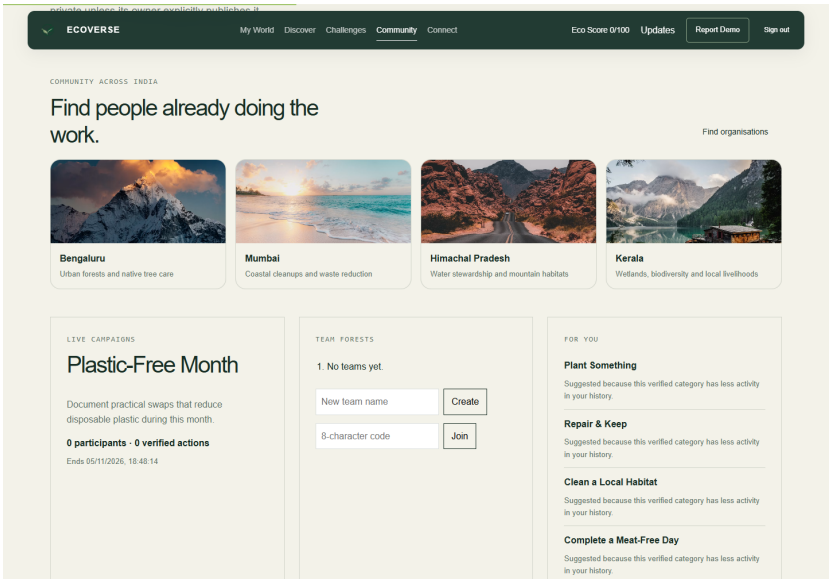


Figure 5. Community across India section. The stock photographs are illustrative and are not verified photographs of the named locations.

Connect provides a separate directory of environmental groups, with search by organization, cause and city. It exposes official volunteering and contact links. The India discovery cards identify themes associated with Bengaluru, Mumbai, Himachal Pradesh and Kerala, while directory details help a user continue to an organization's own website.

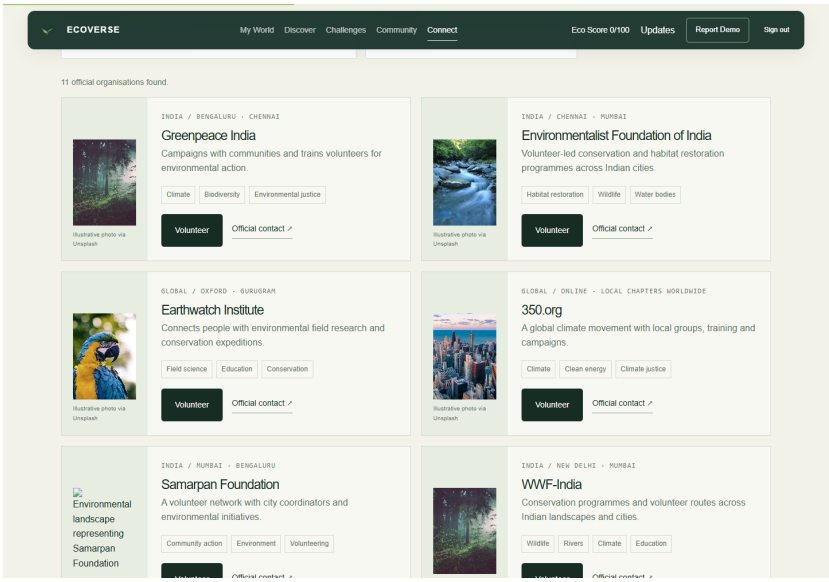


Figure 6. Organization directory served by the Connect API. Listed links do not imply a partnership or guaranteed volunteer availability.

Content quality

Illustrative photographs should not be treated as geographic evidence. Before submission or wider publication, mismatched city imagery should be replaced with verified local photographs or neutral category images. Organization eligibility and current opportunities should be checked on the official linked site.

3. Backend architecture and technology stack

The browser renders the interface and requests structured data from Express through relative /api routes. Express validates input, identifies the signed-in user, checks access and reads or updates MySQL records. JSON responses drive loading, success and error states in the browser. This separation allows visual changes without moving reward calculation or authorization into client-side code.

Layer / technology	Role in the application
HTML5 and CSS3	Semantic multipage layouts, shared navigation, responsive grids, forms and status indicators.
JavaScript ES modules	Page-specific behavior, DOM rendering, search, form submission and asynchronous API requests.
Vite	Development server with API forwarding, shared asset processing and production frontend build.
Node.js and Express	HTTP server, route handlers, authentication, review workflow and application rules.
MySQL and mysql2	Relational persistence, parameterized queries, pooled connections and database transactions.
express-session	Server-side sessions backed by a MySQL session store; cookies identify the session.
Three.js / WebGL	Digital ecosystem rendering with trees, terrain and wildlife assets.
GSAP and Lenis	Animation and scrolling behavior in the interface.
Zod, Helmet and rate limiting	Request validation, security headers and limits on sensitive operations.
Node crypto and Sharp	Salted scrypt password hashing, image hashing and processing of submitted proof.
Nodemailer	Optional email integration when SMTP credentials are configured.
Node test runner / Playwright	Automated logic and API tests, plus browser-based testing tools.

3.1 Request and response sequence

For example, starting a challenge sends its identifier to an authenticated API route. The server validates the identifier and checks whether the user can create an attempt. It stores the attempt before returning a response. The page then changes from an available action to an in-progress workflow. A failure response is presented as an error message rather than treated as a successful action.

3.2 Application organization

Page entry points include index.html, dashboard.html, challenges.html, community.html and connect.html. Shared behavior is centralized in js/shell.js and js/api.js. Backend responsibilities are divided across modules such as app.js, verification.js, community.js and organizations.js. The database schema and migration code define persistent structures. This organization keeps concerns identifiable during testing and maintenance.

4. Database design and backend workflow

The relational model separates users, challenge definitions, attempts and rewards. A challenge is a reusable activity template; a completion record represents one user attempt. Keeping these separate preserves the history of participation while allowing the catalog to grow. Foreign keys connect records, and unique constraints help prevent duplicate reward entries.

Table / group	Purpose and relationship
users; sessions	Identity, password hash and role; session records retain authenticated state.
challenges	Catalog entries, category, difficulty, points, instructions and proof requirements.
challenge_completions	User-to-challenge attempts, status, timestamps, proof reference and review metadata.
point_transactions	Reward ledger; unique completion references protect against duplicate awards.
submission_reviews	Reviewer identity, decision, reason and the evidence snapshot for review history.
forest_progress	Per-user growth totals and configured forest / wildlife unlock values.
badges; user_badges	Badge definitions and the association recording which user earned each badge.
user_extras; quiz_results	Additional per-user state and topic-level learning results.
campaigns; campaign_challenges	Time-bounded collective activity and its associated challenges.
teams; team_members	Named groups, join codes, ownership and membership.
community_posts; reactions; comments	Optional publication of approved actions and associated interaction records.
notifications; review_assignments	User updates and reviewer work allocation.

4.1 Relationships and integrity

A user can create multiple challenge attempts, and a challenge can be attempted by many users. Each attempt can have review history and a linked reward transaction. Users and badges are related through user_badges; teams and users through team_members. The point ledger links rewards to their source, making the displayed score traceable to approved records rather than a number supplied by the browser.

4.2 Evidence processing

Submitting proof changes a valid attempt into a pending review. The backend processes the image with Sharp, computes an image hash and stores the proof reference with the description and optional location. Review rules inspect ownership, eligibility and status before updates are allowed. Approval and reward changes use database transactions so a failed operation can roll back instead of leaving a partial reward.

4.3 Organization directory storage

The current Connect directory is curated server-side data in server/organizations.js and is returned through an API. It is not currently an organization-management database module. A future administration interface could persist and review directory changes, track verification dates and remove outdated contact routes.

5. Complete project flow

The main workflow follows a user from discovery to an evidence-based reward. An administrator reviews the submission. A rejected submission returns to correction; an approved submission updates progression. Community and organization discovery remain accessible through navigation and do not require a reward to be earned first.

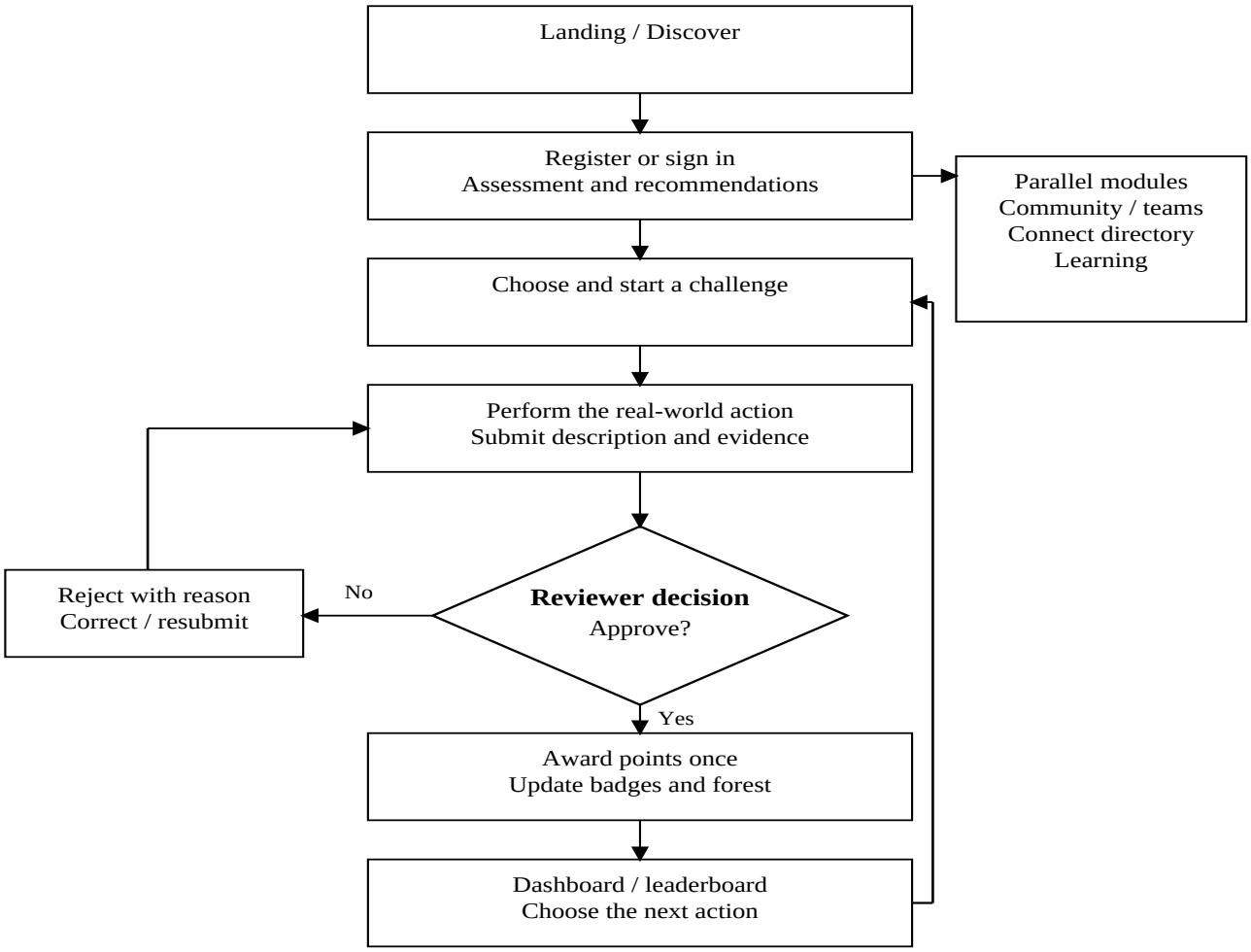


Figure 7. EcoVerse user flow, review decision and repeat-action loop.

5.1 Roles and responsibility

Visitors can inspect public information and organization listings. Members maintain a personal activity record, start challenges, submit proof and participate in community functions. Administrators access protected review operations. The server checks roles independently of the visible navigation, so hiding or showing a button does not determine permission.

5.2 Verification boundary

Approval records a human review decision within EcoVerse. It is not a scientific guarantee that every claimed environmental benefit occurred. Review history, duplicate-image signals and ownership checks improve traceability while keeping the distinction between evidence review and independent certification clear.

6. Security, testing and results

6.1 Security implementation

Passwords are stored as salted scrypt hashes. Server-side sessions use cookies with HttpOnly and SameSite settings, and production enables secure cookies. Request schemas validate expected data, database calls use bound parameters, and role checks protect administrative operations. Rate limits reduce repeated authentication and submission attempts. Origin checks restrict state-changing browser requests to configured origins.

Proof access is checked against the owner or administrator. The community photo route additionally checks that the related action is approved and that photo sharing was selected. These controls reduce accidental exposure of evidence; deployment must still supply correct secrets, HTTPS and durable storage.

6.2 Verification evidence

Check	Recorded result / scope
Production build	Vite build completed successfully before the frontend deployment.
Automated suite	All 19 tests passed during deployment preparation after local MySQL was started.
API workflow	Tests cover verification, fraud controls, transaction rollback and associated learning/workshop behavior.
Community workflow	Tests cover campaigns, teams, notifications, public forest and account privacy.
Progression and assets	Tests cover thresholds, habitat behavior, birds, butterflies, woodland assets and growth behavior.
Frontend hosting	Home, Challenges, Community, Connect and the logo returned HTTP 200 after Vercel deployment.
Report screenshots	Captured from the local app with a temporary database and demonstration account; not production-user data.

6.3 Interpretation of results

Passing automated tests establishes that the tested scenarios behaved as expected. It does not prove that every device, upload or network condition has been covered. Screenshot capture confirms that the selected local pages rendered, but it is not a complete accessibility or visual regression audit. Browser testing on mobile widths and manual review of the three-dimensional scene remain useful before a public release.

6.4 Remaining quality work

The Vercel build reported dependency vulnerabilities during installation. Their impact should be assessed and compatible updates tested before final public backend deployment. The main world bundle also produces a size warning, suggesting a future opportunity to defer heavy scene code. Neither issue should be hidden by describing the current system as fully production-ready.

6.5 Suggested viva demonstration

Open the local application, register a fresh account and inspect its zero-point dashboard. Search for a challenge, start it and submit evidence. Show that the attempt is pending before review. Approve it through an administrator account, then revisit progress and badges. Finally, show team participation and the searchable Connect directory. This sequence demonstrates both interface behavior and backend state changes.

7. Execution and deployment

7.1 Local execution

From the project folder, install dependencies, start the configured local MySQL instance and run the application. Database initialization is required for a new environment. The db:local helper depends on the bundled local installation; it is not a universal MySQL installer. Existing project data should be retained rather than reset for demonstrations.

Command	Purpose
npm install	Install project dependencies.
npm run db:local	Start the configured local database where supported.
npm run db:init	Initialize or migrate local schema; this command also supports legacy import.
npm start	Build the frontend and start Express; local URL is http://localhost:3001.
npm test	Run the automated test suite with a working test database connection.

7.2 Deployment status as of 6 October 2026

The frontend is deployed on Vercel at https://ecoverse-six-blue.vercel.app. The latest deployment configuration is committed on the main branch. Online backend and database setup is incomplete: live authentication, persisted challenge activity and proof uploads must not be presented as fully connected. Local functionality has been exercised with MySQL.

The intended deployment uses Vercel for frontend assets and Render for Express, with an external hosted MySQL database. An API rewrite will connect the frontend to the backend after the Render URL is available. Secure environment variables, a correctly configured application origin and persistent proof storage are still required. Free Render storage is ephemeral, so uploaded evidence needs a separate durable storage solution.

8. Future scope and conclusion

Practical next steps include completing hosting, moving evidence to durable object storage, reviewing dependency updates and improving image attribution. Product improvements can then include richer team events, moderated organization updates and transparent calculation notes for estimated impact. Any environmental estimate should document its assumptions and remain separate from the count of reviewed actions.

Conclusion

EcoVerse demonstrates how a web application can connect sustainable action, review and personal motivation. Its strongest technical contribution is the complete local workflow from authenticated challenge participation to reviewed evidence, reward transactions and visible progress. Community and organization discovery broaden the project beyond an individual score. The current implementation provides a foundation for a final-year demonstration while the report clearly identifies the remaining deployment and quality work.

Source basis and documentation

Implementation references: package.json; server/schema.sql; server/app.js; server/verification.js; server/progression.js; server/community.js; server/organizations.js; js/dashboard.js; tests/; DEPLOYMENT.md. These local sources were inspected for this report. Screenshots were captured on 6 October 2026 using a temporary demonstration account.

Repository: github.com/Hazardo911/Ecoverse. Further reading: nodejs.org/docs; expressjs.com; dev.mysql.com/doc; vite.dev; threejs.org/docs.